

TÀI LIỆU VẬN HÀNH RALLI AI

Contents

TÀI LIỆU VẬN HÀNH HỆ THỐNG RALLI CHATBOT.....	5
0. TÀI LIỆU QUẢN LÝ DỰ ÁN.....	5
0.1. Tổng quan hệ thống (System Overview).....	5
0.2. Scope & Deliverables.....	6
0.3. Danh mục tài sản (Asset List).....	7
2. KIẾN TRÚC VÀ SƠ ĐỒ THỰC TẾ.....	8
2.1. Kiến trúc hệ thống.....	8
1.2. Các thành phần chính.....	9
1.3. Phân chia các Module trong hệ thống.....	10
1.3.4. Services (Dịch vụ Tích hợp).....	11
1.3.5. Multi Agent System.....	11
1.3.6. Task Queue (Hàng đợi Công việc).....	12
1.3.7. Memories/Database (Hệ thống Lưu trữ).....	12
1.3.8. Metrics, Weight & Bias Custom Telemetry.....	12
1.3.9. Domain Tools (Công cụ Chuyên ngành).....	12
2. CÀI ĐẶT VÀ TRIỂN KHAI.....	13
2.1. Yêu cầu hệ thống.....	13
2.2. Cài đặt dependencies.....	13
2.3. Cấu hình môi trường.....	13
#MongoDB.....	13
#API Keys.....	14
#BCI Credentials.....	14
#Zalo API.....	14
#Telegram.....	14
#Google Drive.....	14
#CRM.....	14
2.4. Khởi động hệ thống.....	14
# Khởi động docker.....	14
# Chạy backend.....	15
# Khởi động toàn bộ hệ thống.....	15
# Chỉ khởi động backend.....	15
# Restart hệ thống.....	15
# Dừng hệ thống.....	15
2.5. Cập nhật code mới.....	15

3. QUẢN LÝ SCHEDULER.....	16
3.1. DataFetchScheduler.....	16
3.2. NotificationScheduler.....	17
3.3. DriveUploaderScheduler.....	17
3.4. CRMScheduler.....	18
3.5. LogCleanupScheduler.....	19
3.6. Quản lý Scheduler.....	19
4. QUẢN LÝ DATABASE.....	20
4.1. MongoDB Collections.....	20
4.2. Kết nối Database.....	24
Trong config.py.....	24
Kết nối async.....	24
Kiểm tra qua API.....	24
4.4. Monitoring Database.....	24
Xem thống kê database.....	24
Xem kích thước collections.....	24
Xem index usage.....	25
5. QUẢN LÝ LOGS.....	25
5.1. Cấu trúc log.....	25
5.2. API quản lý logs.....	25
5.3. Monitoring logs.....	25
Sử dụng script có sẵn.....	25
Hoặc xem trực tiếp.....	25
Xem logs cụ thể.....	25
Export logs từ container về máy local.....	26
Export và pull logs (bao gồm cả việc tải về máy).....	26
Tìm kiếm lỗi trong logs.....	27
Tìm kiếm theo nguồn.....	27
Thống kê logs.....	27
6. QUẢN LÝ NGƯỜI DÙNG.....	27
6.1. Tạo admin đầu tiên.....	27
6.2. Quản lý qua API.....	27
6.3. Phân quyền.....	27
7. MONITORING VÀ HEALTH CHECK.....	28
7.1. Health Check API.....	28
7.2. Metrics quan trọng.....	28
7.3. Alerting.....	28
8. BẢO MẬT.....	28
8.1. Authentication & Authorization.....	28

8.2. Network Security.....	29
Cấu hình CORS trong main.py.....	29
8.3. Data Security.....	30
8.4. Security Best Practices.....	32
10. TROUBLESHOOTING.....	33
10.1. Common Issues.....	33
10.2. Debug Commands.....	33
Kiểm tra processes.....	33
Kiểm tra ports.....	33
Kiểm tra logs.....	33
Kiểm tra database.....	33
11. MAINTENANCE.....	33
11.1. Daily Tasks.....	33
11.2. Weekly Tasks.....	34
11.3. Monthly Tasks.....	34
12. SCALING.....	34
12.1. Horizontal Scaling.....	34
12.2. Vertical Scaling.....	34
13. DISASTER RECOVERY.....	34
13.1. RTO/RPO (Recovery Time Objective / Recovery Point Objective).....	34
13.2. Recovery Steps.....	35
14. GOOGLE APPS SCRIPTS.....	36
14.1. Tổng quan AppScripts.....	36
14.2. Script đồng bộ dữ liệu Zalo.....	36
14.3. Script đồng bộ dữ liệu sản phẩm.....	36
14.4. Script đồng bộ Catalogs.....	37
14.5. Cách sử dụng AppScripts.....	37
14.6. API Endpoints được sử dụng.....	38
15. KIẾN TRÚC HỆ THỐNG CHI TIẾT.....	38
15.1. Sơ đồ kiến trúc hệ thống.....	38
16. SOURCE CODE & REPOSITORY.....	39
16.1. Cấu trúc thư mục.....	39
16.2. Naming Convention.....	41
16.3. Hướng dẫn Build & Run.....	42
1. Clone repository.....	42
2. Install dependencies.....	42
3. Setup environment.....	42
Edit .env with your configuration.....	42
4. Start MongoDB.....	42

5. Run application.....	42
1. Build Docker images.....	42
2. Start services.....	42
3. Check logs.....	43
1. Deploy with production compose.....	43
2. Setup SSL.....	43
3. Monitor services.....	43
16.4. Coding Guidelines.....	43
17. API DOCUMENTATION.....	44
17.1. Swagger UI.....	44
17.2. Authentication.....	44
17.3. Rate Limiting.....	44
17.4. Error Codes.....	44
18. DATABASE DOCUMENTATION.....	45
18.1. Mô hình dữ liệu (ERD).....	45
18.2. Database Initialization.....	46
Initialize database.....	47
Create admin user.....	47
18.3. Indexes & Optimization.....	47
18.4. Data Retention Policy.....	48
19. INFRASTRUCTURE / DEVOPS.....	48
19.1. Docker Configuration.....	48
19.2. CI/CD Pipeline.....	50
19.3. Backup & Restore Strategy.....	51
Backup MongoDB.....	51
Restore MongoDB.....	51
19.4. Monitoring & Logging.....	51

TÀI LIỆU VẬN HÀNH HỆ THỐNG RALLI CHATBOT

0. TÀI LIỆU QUẢN LÝ DỰ ÁN

0.1. Tổng quan hệ thống (System Overview)

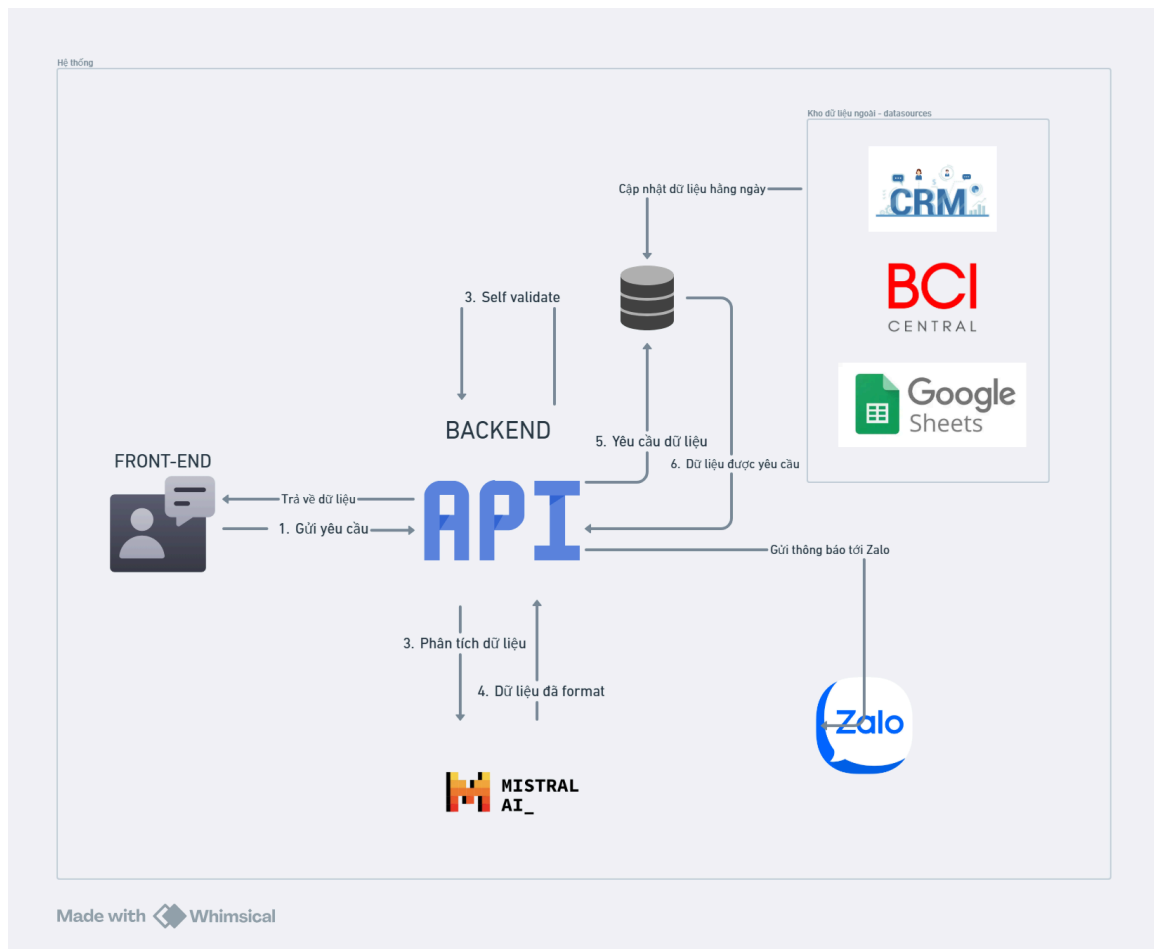
Mục tiêu dự án:

- Xây dựng hệ thống AI Chatbot hỗ trợ báo giá thông tin sản phẩm, thông tin dự án (BCI) và hệ thống CRM
- Tích hợp với các hệ thống hiện có: BCI, Zalo, Google Sheets, Google Drive
- Tự động hóa quy trình báo giá sản phẩm, quản lý dự án, và báo cáo CRM

Phạm vi dự án:

- **Frontend:** Giao diện web responsive cho người dùng cuối
- **Backend:** API server với FastAPI và Python
- **Database:** MongoDB để lưu trữ dữ liệu
- **AI Integration:** Mistral AI cho xử lý ngôn ngữ tự nhiên
- **External Integrations:** Zalo, BCI, Google Services
- **Scheduling:** Hệ thống tự động đồng bộ dữ liệu và gửi báo cáo

Sơ đồ tổng quan:



0.2. Scope & Deliverables

Source Code:

- Repository: rangdong-chatbot
- Main application: main.py
- Core modules: src/core/, src/services/, src/schedulers/
- UI components: ui/ (HTML, CSS, JavaScript)
- Configuration: config.py, requirements.txt

API & Documentation:

- REST API với FastAPI

- Swagger UI documentation tại /docs
- API endpoints cho tất cả chức năng
- Postman collection (có thể tạo từ Swagger)

Database:

- MongoDB collections: users, projects, contacts, crm_data, logs
- Database initialization scripts: src/setup_db.py
- Indexes và optimization queries

Infrastructure:

- Docker containers: Dockerfile.backend, Dockerfile.frontend
- Docker Compose: docker-compose.yml, deploy/docker-compose.prod.yml
- Nginx configuration: nginx.conf
- SSL certificates và HTTPS setup

Documentation:

- Hướng dẫn sử dụng: huong_dan_su_dung.md và huong_dan_su_dung.docx
- Tài liệu vận hành: tai_lieu_van_hanh.md và tai_lieu_van_hanh.docx
- Troubleshooting guide: cac_loi_phat_sinh.md và cac_loi_phat_sinh.docx
- API documentation: Swagger UI

0.3. Danh mục tài sản (Asset List)

Repository & Code:

- Git Repository: rangdong-chatbot
- Main Branch: main
- Development Branch: develop
- Feature Branches: feature/

Server & Infrastructure:

- Production Server: ralliai.rangdong.com.vn
- Ports: 9000 (Backend), 9001 (Frontend)

- SSL Certificate: Let's Encrypt
- Domain: ralliai.rangdong.com.vn

Cloud Services:

- MongoDB: Cloud hoặc self-hosted
- Google Cloud: Drive API, Sheets API
- Mistral AI: API service
- Zalo Platform: OA (Official Account)

API Keys & Credentials:

- Mistral API Key: MISTRAL_API_KEY
- BCI Credentials: BCI_USERNAME, BCI_PASSWORD
- Zalo Tokens: ZALO_ACCESS_TOKEN, ZALO_REFRESH_TOKEN
- Google Service Account: JSON key file
- MongoDB: Connection string và credentials

External Accounts:

- Google Sheets:
 - o Sản phẩm:
https://docs.google.com/spreadsheets/d/1UWH4a7Efw2Jb5KWcC4xdYldRTX2noe_SdSep8FRaGVY/edit?usp=sharing
 - o Zalo contacts:
https://docs.google.com/spreadsheets/d/1o4PAWEOWKlcRfTaPBz0RlkFUPsFVScKRIjuej78PH0s/edit?usp=drive_link
 - o Catalogs:
https://docs.google.com/spreadsheets/d/1SWk7Q2gaM4_wqXhquq77j28waLISGlvhS9tbp23X3ZQ/edit?usp=drive_link

2. KIẾN TRÚC VÀ SƠ ĐỒ THỰC TẾ

2.1. Kiến trúc hệ thống

- **Frontend:** HTML/CSS/JavaScript (Vanilla)
- **Backend:** FastAPI (Python 3.10+) với Swagger UI
- **Database:** MongoDB

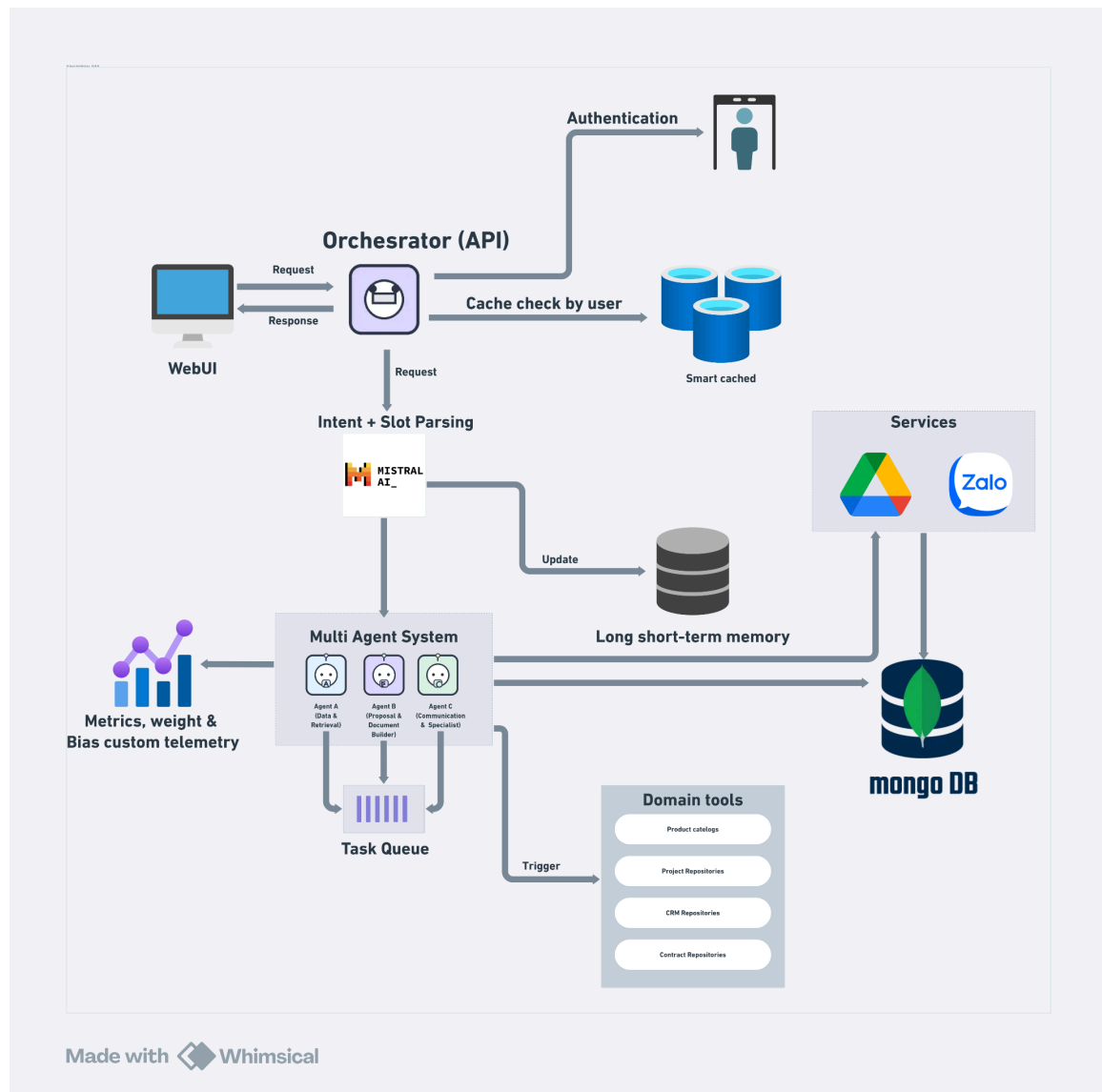
- **Scheduler:** APScheduler
- **External APIs:** Zalo, BCI, CRM, Google Drive, Telegram
- **API Documentation:** Swagger UI tại /docs

1.2. Các thành phần chính

Phân loại các thành phần chính theo folder

- **Main Application:** main.py
- **Authentication:** src/auth/
- **Schedulers:** src/schedulers/
- **Services:** src/services/
- **Repositories:** src/repositories/
- **UI:** ui/

1.3. Phân chia các Module trong hệ thống



Sơ đồ AI Agent Architecture

1.3.1. WebUI

- Thành phần: Giao diện web người dùng (HTML/js/css)
- Chức năng:
 - Cung cấp giao diện trò chuyện trực tiếp với chatbot
 - Hiển thị lịch sử cuộc hội thoại
 - Quản lý phiên làm việc của người dùng
- Đặc điểm:
 - Giao diện thân thiện, responsive
 - Hỗ trợ real-time messaging

- Tích hợp với Orchestrator thông qua API

1.3.2. Orchestrator (Trung tâm điều phối)

- Vai trò: Bộ điều phối trung tâm của toàn bộ hệ thống (API)
- Chức năng:
 - Nhận request từ WebUI
 - Phân tích ý định và nội dung của người dùng (Intent + Slot Parsing)
 - Định tuyến request đến Multi-Agent System phù hợp
 - Tổng hợp và trả về response cuối cùng
- Đặc điểm:
 - Xử lý đồng thời nhiều request
 - Quản lý luồng dữ liệu giữa các module
 - Tối ưu hóa hiệu suất hệ thống

1.3.3. Intent + Slot Parsing

- Thành phần: Module phân tích ngôn ngữ tự nhiên (Mistral AI)
- Chức năng:
 - Intent Recognition: Nhận diện ý định của người dùng (hỏi giá, tư vấn sản phẩm, CRM, projects...)
 - Slot Extraction: Trích xuất thông tin chi tiết (tên sản phẩm, model, thông số kỹ thuật...)
 - Chuẩn hóa dữ liệu đầu vào cho Multi-Agent System
- Đặc điểm:
 - Sử dụng Mistral AI 7b-v3
 - Hỗ trợ tiếng Việt và tiếng Anh
 - Học từ lịch sử hội thoại để cải thiện độ chính xác

1.3.4. Services (Dịch vụ Tích hợp)

- Google Drive Integration:
 - Đồng bộ tài liệu sản phẩm từ Google Drive
 - Hỗ trợ cập nhật lúc người dùng cần
 - Kiểm soát quota, backoff, theo dõi lỗi I/O.
- Zalo Integration:
 - Adapter gửi message/file qua Zalo OA (định dạng template, người nhận)
 - Trả trạng thái gửi/thời gian/ID tin nhắn

1.3.5. Multi Agent System

- Agent A: Data & Retrieval
 - Chuyên về thông tin sản phẩm (TSKT, giá cả, tính năng)
 - Tư vấn lựa chọn sản phẩm phù hợp

- Cập nhật tồn kho/thay đổi giá real-time
- Agent B: Proposal & Document Builder
 - Tổng hợp dữ liệu từ Agent A để tính giá/chiết khấu
 - Tạo báo giá/bảng so sánh dạng Excel/PDF theo template chuẩn; upload Drive và trả link.
- Agent C: Communication & Specialist
 - Gửi tài liệu/kết quả qua Zalo OA, theo dõi trạng thái giao hàng.
 - Ghi activity (đính kèm link Drive, người nhận, thời gian), tạo nhắc việc follow-up.
- Đặc điểm chung:
 - Mỗi agent có knowledge base chuyên biệt
 - Làm việc song song và độc lập
 - Kết quả được tổng hợp bởi Task Queue

1.3.6. Task Queue (Hàng đợi Công việc)

- Chức năng:
 - Quản lý và phân phối task
 - Đảm bảo dữ liệu truy cập ổn định
 - Theo dõi trạng thái xử lý của từng task
- Đặc điểm:
 - Hỗ trợ priority queue
 - Retry mechanism khi task fail
 - Monitoring và logging chi tiết

1.3.7. Memories/Database (Hệ thống Lưu trữ)

- MongoDB + SmartCached:
 - Chat Histories: Lưu trữ lịch sử hội thoại đầy đủ
 - User Profiles: Thông tin và preferences của người dùng
 - Session Management: Quản lý phiên làm việc
 - Analytics Data: Dữ liệu phân tích hiệu suất hệ thống

1.3.8. Metrics, Weight & Bias Custom Telemetry

- Performance Monitoring:
 - Theo dõi response time, success rate
 - Monitoring resource usage (CPU, memory, database)
 - Alert system khi có anomaly
- Quality Assurance:
 - Đánh giá chất lượng câu trả lời
 - Feedback loop để cải thiện liên tục
 - Memories ghi nhớ thông tin để hiểu mong muốn người dùng.
 - Cảnh báo: error rate, p95 latency, cost/request, retriever latency

1.3.9. Domain Tools (Công cụ Chuyên ngành)

- Product Catalogs:
 - Nhận diện được các ký hiệu đặc biệt sử dụng chuyên biệt cho Ralli
 - Các từ khóa viết tắt hoặc từ ngữ của riêng hệ thống Rạng Đông
- Project Repositories (BCI):
 - Dữ liệu BCI lấy trực tiếp từ BCI Central mỗi ngày
 - Tích hợp đầy đủ vào hệ thống Ralli AI
 - Quản lý thông tin BCI và dễ dàng truy vấn
- CRM Repositories:
 - Đầy đủ thông tin CRM lấy từ Portal
 - Dữ liệu cập nhật với mỗi 6 tiếng để đảm bảo đầy đủ.
 - Tích hợp đầy đủ vào hệ thống Ralli AI
 - Quản lý thông tin và dễ dàng truy vấn
- Contract Repositories:
 - Dữ liệu các đầu môi quản lý thông tin BCI cần gửi về Zalo
 - Sync trực tiếp từ Excel và settings linh động
 - Tự động lên lịch và gửi dữ liệu lúc cần.

2. CÀI ĐẶT VÀ TRIỂN KHAI

2.1. Yêu cầu hệ thống

- **OS:** Linux/Windows

- **Python:** 3.10+

- **MongoDB:** 4.0+

- **RAM:** Tối thiểu 2GB

- **Disk:** Tối thiểu 20GB

2.2. Cài đặt dependencies

pip install -r requirements.txt

2.3. Cấu hình môi trường

Tạo file .env với các biến:

#MongoDB

MONGODB_URI=mongodb://admin:password@localhost:27017

MONGODB_DB=rangdong

MONGODB_COLLECTION=projects

#API Keys

MISTRAL_API_KEY=your_mistral_key

BUCKET_APP=your_bucket_app

#BCI Credentials

BCI_USERNAME=your_bci_username

BCI_PASSWORD=your_bci_password

#Zalo API

ZALO_IMEI=your_zalo_imei

ZALO_PHONE=your_zalo_phone

ZALO_PASSWORD=your_zalo_password

ZALO_OA_APP_ID=your_zalo_oa_app_id

ZALO_OA_SECRET_KEY=your_zalo_oa_secret_key

#Telegram

TELEGRAM_BOT_TOKEN=your_telegram_bot_token

TELEGRAM_CHAT_ID=your_telegram_chat_id

#Google Drive

PROJECT_FOLDER_ID=your_project_folder_id

DAILY_FOLDER_ID=your_daily_folder_id

SAP_LINK_ID=your_sap_link_id

#CRM

CRM_USERNAME=your_crm_username

CRM_PASSWORD=your_crm_password

2.4. Khởi động hệ thống

Development:

Khởi động docker

Docker-compose up -d

Chạy backend

Uvicorn main:app --reload --port 9014

Production (Docker):

Khởi động toàn bộ hệ thống

./deploy/start.sh

Chỉ khởi động backend

docker-compose -f deploy/docker-compose.prod.yml up -d

Restart hệ thống

./deploy/restart.sh

Dừng hệ thống

./deploy/stop.sh

Truy cập hệ thống:

- API Documentation (Swagger): <https://ralliai.rangdong.com.vn:9000/docs>

- Frontend: <https://ralliai.rangdong.com.vn:9001>

- Backend API: <https://ralliai.rangdong.com.vn:9000>

2.5. Cập nhật code mới

Quy trình update code mới, thực hiện chạy theo các bước sau đây:

1. Restart hệ thống:

Chạy script restart để cập nhật code mới

./deploy/restart.sh

2. Kiểm tra hệ thống hoạt động:

Kiểm tra trạng thái scheduler

```
curl -X GET "https://ralliai.rangdong.com.vn:9000/scheduler/health-check"
```

Kiểm tra logs

```
./deploy/watch_log.sh
```

Lưu ý quan trọng:

- Luôn cập nhật dữ liệu trước khi restart để đảm bảo hệ thống có dữ liệu mới nhất
- Script restart.sh sẽ tự động build lại Docker image và restart container
- Quá trình restart mất khoảng 2-3 phút, hệ thống sẽ tạm thời không khả dụng

3. QUẢN LÝ SCHEDULER

3.1. DataFetchScheduler

Chức năng: Đồng bộ dữ liệu dự án từ BCI

Tần suất: Hàng ngày lúc 6:00 AM (Vietnam time)

Dữ liệu đồng bộ:

- Danh sách dự án mới từ BCI
- Thông tin chi tiết dự án (tên, địa điểm, ngày bắt đầu/kết thúc)
- Trạng thái dự án và người phụ trách
- Cập nhật database MongoDB

Cấu hình retry:

- Max retries: 3
- Retry delay: 300s (5 phút)
- Backoff multiplier: 2.0

API endpoints:

- POST /trigger-data-fetch: Chạy thủ công
- GET /scheduler/retry-status: Kiểm tra trạng thái retry
- POST /scheduler/force-data-fetch: Bắt buộc chạy ngay

3.2. NotificationScheduler

Chức năng: Quản lý thông báo và báo cáo BCI

Tần suất:

- Cập nhật dữ liệu daily: Hàng ngày lúc 6:00 AM (Vietnam time)
- Gửi báo cáo BCI: Theo lịch được cấu hình hoặc khi có tín hiệu

Dữ liệu cập nhật/thông báo:

- Dữ liệu daily: Danh sách dự án mới từ BCI, thông tin chi tiết dự án
- Báo cáo BCI: File Excel báo cáo dự án theo định kỳ
- Thông báo Zalo: Gửi báo cáo BCI đến các đầu mối qua Zalo
- Thông báo Telegram: Thông báo tình trạng hệ thống
- Upload Drive: Tự động upload file báo cáo lên Google Drive

Cấu hình lịch gửi:

- Có thể cấu hình qua Google Sheets (lịch gửi Zalo)
- Hỗ trợ: Không lặp, Hàng ngày, Hàng tuần, Hàng tháng
- Thời gian gửi: Tùy chỉnh theo nhu cầu

API endpoints:

- POST /trigger-notification: Chạy thủ công
- POST /scheduler/config: Cấu hình lịch gửi báo cáo

3.3. DriveUploaderScheduler

Chức năng: Quản lý file và đồng bộ với Google Drive

Tần suất: Hàng ngày lúc 6:00 AM (Vietnam time)

Dữ liệu xử lý:

- File báo cáo dự án: Excel files từ DataFetchScheduler
- File báo cáo CRM: Excel files từ CRMScheduler
- File logs: Log files được nén và lưu trữ
- File cấu hình: Backup các file cấu hình quan trọng

Nhiệm vụ:

- Upload file báo cáo dự án lên Drive
- Tạo thư mục theo ngày tháng (YYYY/MM/DD)
- Đồng bộ file với các thư mục được cấu hình
- Cập nhật link chia sẻ file
- Quản lý quyền truy cập file

API endpoints:

- POST /trigger-drive: Chạy thủ công
- POST /trigger-drive-upload: Upload file cụ thể

3.4. CRMScheduler

Chức năng: Đồng bộ dữ liệu CRM từ Google Sheets

Tần suất: Mỗi 6 giờ (4 lần/ngày: 0:00, 6:00, 12:00, 18:00 Vietnam time)

Dữ liệu đồng bộ:

- Danh sách khách hàng: Tên, số điện thoại, email, công ty, chức vụ
- Thông tin liên hệ Zalo: User ID Zalo, khu vực, tỉnh thành
- Từ viết tắt nội bộ (Catalogs): Từ lóng, thuật ngữ chuyên môn
- Từ đặc biệt (Specials): Từ khóa đặc biệt cho sản phẩm
- Lịch gửi Zalo: Cấu hình thời gian gửi báo cáo
- Cấu hình scheduler: Thời gian, tần suất gửi báo cáo

Nguồn dữ liệu:

- Google Sheets Zalo: 1o4PAWEOWKlcRfTaPBz0RIkFUPsFVScKRljuej78PH0s
- Google Sheets Catalogs: 1SWk7Q2gaM4_wqXhquq77j28waLISGlvhS9tbp23X3ZQ

API endpoints:

- POST /trigger-crm-fetch: Chạy thủ công
- POST /contacts/sync: Đồng bộ danh sách liên hệ
- POST /scheduler/config: Cập nhật cấu hình lịch gửi

3.5. LogCleanupScheduler

Chức năng: Quản lý và dọn dẹp log hệ thống

Tần suất: Hàng ngày lúc 2:00 AM (Vietnam time)

Dữ liệu xử lý:

- User logs: Nhật ký hoạt động người dùng
- Mistral logs: Log tương tác với AI
- Product logs: Log xử lý sản phẩm
- Scheduler logs: Log các tác vụ tự động
- Zalo logs: Log tích hợp Zalo
- Common logs: Log chung của hệ thống
- Warning logs: Log cảnh báo và lỗi

Nhiệm vụ:

- Xóa log cũ hơn 30 ngày (mặc định)
- Nén log cũ để tiết kiệm dung lượng
- Báo cáo dung lượng log đã dọn dẹp
- Giữ lại log quan trọng (error, warning)
- Tối ưu hóa hiệu suất đọc/ghi log

API endpoints:

- POST /logs/cleanup: Chạy thủ công với tham số days_to_keep
- GET /logs/cleanup-status: Kiểm tra trạng thái dọn dẹp

3.6. Quản lý Scheduler

Kiểm tra trạng thái tất cả scheduler:

```
curl -X GET "https://ralliai.rangdong.com.vn:9000/scheduler/health-check"
```

Xem trạng thái chi tiết:

```
curl -X GET "https://ralliai.rangdong.com.vn:9000/scheduler/detailed-status"
```

Cập nhật cấu hình retry:

```
curl -X POST "https://ralliai.rangdong.com.vn:9000/scheduler/update-retry-config" \  
-H "Content-Type: application/json" \  
-d '{"max_retries": 5, "retry_delay": 600, "backoff_multiplier": 1.5}'
```

4. QUẢN LÝ DATABASE

4.1. MongoDB Collections

4.1.1. users

- Mục đích: Quản lý người dùng hệ thống

- Cấu trúc:

json

```
{  
  "_id": "ObjectId",  
  "username": "string (unique)",  
  "email": "string (unique)",  
  "hashed_password": "string",  
  "role": "ADMIN|MANAGER|SPECIALIST|MEMBER",  
  "is_active": "boolean",  
  "created_at": "datetime",  
  "last_login": "datetime"  
}
```

4.1.2. projects

- Mục đích: Lưu trữ dữ liệu dự án từ BCI

- Cấu trúc:

json

```
{
  "_id": "ObjectId",
  "project_id": "string (unique)",
  "project_name": "string",
  "project_type": "string",
  "project_stage": "string",
  "province": "string",
  "value": "number",
  "company": "string",
  "contact": "string",
  "start_date": "datetime",
  "end_date": "datetime",
  "status": "string",
  "manager": "string",
  "last_update": "datetime",
  "created_at": "datetime"
}
```

4.1.3. contacts

- Mục đích: Danh sách liên hệ CRM

- Cấu trúc:

json

```
{
  "_id": "ObjectId",
```

```
"name": "string",  
"phone": "string",  
"email": "string",  
"company": "string",  
"position": "string",  
"zalo_user_id": "string",  
"region": "string",  
"created_at": "datetime",  
"updated_at": "datetime"  
}
```

4.1.4. crm_data

- Mục đích: Dữ liệu báo cáo CRM

- Cấu trúc:

json

```
{  
  "_id": "ObjectId",  
  "organization_name": "string",  
  "user_name": "string",  
  "period_type": "month|quarter",  
  "period_value": "number",  
  "year": "number",  
  "metrics": {  
    "AI": {  
      "BCI": {
```

```

"approached": {"revenue": "number", "count": "number"},
"care": {"revenue": "number", "count": "number"},
"success": {"revenue": "number", "count": "number"},
"potential": {"revenue": "number", "count": "number"}
},
"Khac": { /* tương tự BCI */ }
},
"CTDA": { /* tương tự AI */ },
"DienApMai": { /* tương tự AI */ }
},
"created_at": "datetime",
"updated_at": "datetime"
}

```

4.1.5. logs

- Mục đích: Nhật ký hoạt động hệ thống

- Cấu trúc:

json

```

{
  "_id": "ObjectId",
  "timestamp": "datetime",
  "level": "INFO|WARNING|ERROR|DEBUG",
  "source": "user|mistral|product|scheduler|zalo|common",
  "message": "string",
  "user_id": "string (optional)",

```

```
"request_id": "string (optional)",  
"details": "object (optional)"  
}
```

4.2. Kết nối Database

Cấu hình kết nối:

Trong config.py

```
MONGODB_URI = "mongodb://admin:password@localhost:27017"  
MONGODB_DB = "rangdong"  
MONGODB_COLLECTION = "projects"
```

Kết nối async

```
client = motor.motor_asyncio.AsyncIOMotorClient(MONGODB_URI)  
db = client[MONGODB_DB]
```

Kiểm tra kết nối:

```
bash
```

Kiểm tra qua API

```
curl -X GET "https://ralliai.rangdong.com.vn:9000/api/status"
```

4.4. Monitoring Database

Kiểm tra hiệu suất:

Xem thống kê database

```
mongosh "mongodb://admin:password@localhost:27017/rangdong" --eval "db.stats()"
```

Xem kích thước collections

```
mongosh "mongodb://admin:password@localhost:27017/rangdong" --eval  
"db.runCommand({dbStats: 1})"
```

Xem index usage

```
mongosh "mongodb://admin:password@localhost:27017/rangdong" --eval  
"db.projects.aggregate([{$indexStats: {}}])"
```

5. QUẢN LÝ LOGS

5.1. Cấu trúc log

- User logs: logs/user_YYYY-MM-DD.log
- Mistral logs: logs/mistral_YYYY-MM-DD.log
- Product logs: logs/product_YYYY-MM-DD.log
- Warning logs: logs/warning_YYYY-MM-DD.log
- Scheduler logs: logs/scheduler_YYYY-MM-DD.log
- Zalo logs: logs/zalo_YYYY-MM-DD.log
- Common logs: logs/common_YYYY-MM-DD.log

5.2. API quản lý logs

- GET /logs/recent: Lấy logs gần đây
- POST /logs/cleanup: Dọn dẹp logs cũ
- GET /logs/cleanup-status: Trạng thái cleanup

5.3. Monitoring logs

Xem logs real-time:

Sử dụng script có sẵn

```
./deploy/watch_log.sh
```

Hoặc xem trực tiếp

```
docker logs -f deploy_backend_1
```

Xem logs cụ thể

```
tail -f logs/common_$(date +%Y-%m-%d).log
```

Export logs:

Export logs từ container về máy local

`./deploy/export_logs.sh`

Export và pull logs (bao gồm cả việc tải về máy)

`./deploy/export_and_pull_logs.sh`

Hướng dẫn sử dụng export logs:

1. Export logs cơ bản:

- Chạy `./deploy/export_logs.sh` để copy logs từ container về thư mục `/usr/ralli/rangdong-chatbot/logs`
- Logs sẽ được lưu theo cấu trúc: `logs/user_YYYY-MM-DD.log`, `logs/common_YYYY-MM-DD.log`, etc.

2. Export và pull logs:

- Chạy `./deploy/export_and_pull_logs.sh` để export và tải logs về máy local
- Script sẽ tự động tạo thư mục logs trên máy local
- Phù hợp khi cần phân tích logs trên máy khác

3. Xem logs đã export:

Xem danh sách logs đã export

`ls -la /usr/ralli/rangdong-chatbot/logs/`

Xem nội dung log cụ thể

`cat /usr/ralli/rangdong-chatbot/logs/common_2024-01-15.log`

Tìm kiếm lỗi:

Tìm kiếm lỗi trong logs

```
grep -i "error\|exception" logs/*.log
```

Tìm kiếm theo nguồn

```
grep -i "scheduler" logs/common_$(date +%Y-%m-%d).log
```

Thống kê logs

```
wc -l logs/*.log
```

Xem logs qua Admin UI:

1. Truy cập **Hệ thống & Logs**
2. Chọn Kéo xuống dưới có thể tìm kiếm logs
3. Sử dụng bộ lọc để tìm kiếm logs cụ thể

6. QUẢN LÝ NGƯỜI DÙNG

6.1. Tạo admin đầu tiên

```
python
```

```
python create_test_admin.py
```

6.2. Quản lý qua API

- GET /users/list: Danh sách người dùng
- POST /users/create: Tạo người dùng mới
- PUT /users/{username}/role: Cập nhật role
- DELETE /users/{username}: Xóa người dùng

6.3. Phân quyền

- ADMIN: Toàn quyền
- MANAGER: Quản lý dự án, báo cáo
- SPECIALIST: Truy cập chức năng chuyên sâu
- MEMBER: Quyền cơ bản

- GUEST: Chỉ xem

7. MONITORING VÀ HEALTH CHECK

7.1. Health Check API

- GET /api/status: Trạng thái tổng quan
- GET /scheduler/health-check: Trạng thái scheduler
- GET /scheduler/detailed-status: Chi tiết trạng thái

7.2. Metrics quan trọng

- CPU Usage: < 80%
- Memory Usage: < 85%
- Disk Usage: < 90%
- Database Connections: < 100
- Response Time: < 2s

7.3. Alerting

- Scheduler failures
- Database connection errors
- High resource usage
- Authentication failures

8. BẢO MẬT

8.1. Authentication & Authorization

8.1.1. JWT Token Configuration

- Secret Key: Được cấu hình trong biến môi trường JWT_SECRET_KEY
- Algorithm: HS256 (mặc định)
- Token Expiration: 120 phút (có thể cấu hình qua JWT_ACCESS_TOKEN_EXPIRE_MINUTES)
- Refresh Token: Không có (cần đăng nhập lại khi hết hạn)

Cấu hình JWT:

env

JWT_SECRET_KEY=your_secret_key_here

JWT_ALGORITHM=HS256

JWT_ACCESS_TOKEN_EXPIRE_MINUTES=120

8.1.2. Password Security

- Hashing: BCrypt với salt rounds
- Minimum Requirements: Không có yêu cầu đặc biệt (có thể thêm)
- Password Reset: Chưa implement

8.1.3. Role-Based Access Control

- ADMIN: Toàn quyền hệ thống
- MANAGER: Quản lý người dùng và xem báo cáo
- SPECIALIST: Xem và tạo báo cáo
- MEMBER: Chỉ xem dữ liệu cơ bản

8.2. Network Security

8.2.1. HTTPS Configuration

- Production: Sử dụng SSL/TLS certificates
- Port: 9000 (HTTPS), 9001 (Frontend)
- Certificate: Let's Encrypt hoặc custom certificates

8.2.2. CORS Configuration

python

Cấu hình CORS trong main.py

```
app.add_middleware(  
  
    CORSMiddleware,  
  
    allow_origins=["https://ralliai.rangdong.com.vn"],  
  
    allow_credentials=True,  
  
    allow_methods=["*"],  
  
    allow_headers=["*"],
```

)

8.2.3. Rate Limiting

- Tại sao cần Rate Limiting:
- Ngăn chặn tấn công DDoS và spam requests
- Bảo vệ server khỏi quá tải khi có quá nhiều request đồng thời
- Ngăn chặn brute force attacks trên API authentication
- Đảm bảo tài nguyên server được phân bổ công bằng cho tất cả người dùng
- API Calls: Hiện tại chưa có rate limiting (khuyến nghị thêm: 100 requests/phút/user)
- Authentication: Chưa có brute force protection (khuyến nghị: 5 lần thử/phút)

8.3. Data Security

8.3.1. Mã hóa dữ liệu nhạy cảm

- Mật khẩu người dùng:
- Sử dụng BCrypt với salt rounds để hash mật khẩu
- Mật khẩu gốc không bao giờ được lưu trữ trong database
- Mỗi mật khẩu có salt riêng, ngay cả khi giống nhau cũng có hash khác nhau
- Không thể reverse engineer từ hash về mật khẩu gốc
- API Keys và tokens:
- Tất cả API keys được lưu trữ trong biến môi trường (.env file)
- Không bao giờ commit API keys vào source code
- JWT tokens được ký bằng secret key mạnh
- Zalo tokens được mã hóa trước khi lưu vào database
- Dữ liệu database:
- Hiện tại chưa có encryption at rest (dữ liệu được mã hóa khi lưu trữ)
- Khuyến nghị: Sử dụng MongoDB encryption at rest cho dữ liệu nhạy cảm
- Kết nối database sử dụng SSL/TLS

8.3.2. Ghi log truy cập và bảo mật

- Sự kiện đăng nhập:
- Ghi lại mọi lần đăng nhập thành công/thất bại
- Lưu thời gian, IP address, user agent
- Phát hiện đăng nhập từ IP lạ hoặc thiết bị mới
- Truy cập API:
- Log tất cả API calls với thông tin user, endpoint, response time
- Phát hiện patterns bất thường trong việc sử dụng API
- Monitor các request có thể là tấn công
- Thất bại bảo mật:
- Ghi lại tất cả failed login attempts với level WARNING
- Alert khi có quá nhiều lần thử đăng nhập từ cùng IP
- Log các lỗi bảo mật và exception

8.3.3. Bảo mật API bên ngoài

- Zalo API:
- Sử dụng OAuth 2.0 với access token và refresh token
- Tokens có thời hạn 3 tháng, cần refresh định kỳ
- Chỉ gửi tin nhắn đến các user ID đã được xác thực
- BCI API:
- Sử dụng Basic Authentication với username/password
- Credentials được lưu trong biến môi trường
- Có retry mechanism khi API không khả dụng
- Google Drive API:
- Sử dụng OAuth 2.0 với service account
- Service account có quyền hạn chế chỉ trên thư mục được chỉ định

- Files được upload với quyền riêng tư, chỉ chia sẻ khi cần

- Mistral AI API:

- Sử dụng API key authentication

- API key được rotate định kỳ

- Có rate limiting để tránh vượt quota

8.3.4. Bảo vệ dữ liệu người dùng

- Dữ liệu cá nhân:

- Thông tin liên hệ được mã hóa khi truyền tải

- Chỉ admin mới có quyền xem thông tin chi tiết người dùng

- Có thể xóa dữ liệu người dùng theo yêu cầu (GDPR compliance)

- Dữ liệu dự án:

- Thông tin dự án được phân quyền theo role

- Chỉ người có quyền mới được xem báo cáo chi tiết

- Dữ liệu được backup định kỳ để tránh mất mát

- Dữ liệu CRM:

- Thông tin khách hàng được bảo vệ nghiêm ngặt

- Chỉ nhân viên có quyền mới được truy cập

- Có audit trail cho mọi thay đổi dữ liệu

8.4. Security Best Practices

8.4.1. Regular Security Updates

- Cập nhật dependencies thường xuyên

- Kiểm tra security vulnerabilities

- Monitor security advisories

8.4.2. Access Control

- Chỉ admin mới có quyền tạo/xóa user

- Token expiration ngắn (2 giờ)

- Không có session persistence

8.4.3. Monitoring & Alerting

- Log tất cả authentication events
- Monitor failed login attempts
- Alert khi có suspicious activities

10. TROUBLESHOOTING

10.1. Common Issues

- Scheduler không chạy: Kiểm tra timezone, restart service
- Database connection failed: Kiểm tra MongoDB service
- API timeout: Kiểm tra network, tăng timeout
- Memory leak: Restart application

10.2. Debug Commands

bash

Kiểm tra processes

ps aux | grep python

Kiểm tra ports

netstat -tlnp | grep 8000

Kiểm tra logs

tail -f logs/common_\$(date +%Y-%m-%d).log

Kiểm tra database

mongo --eval "db.stats()"

11. MAINTENANCE

11.1. Daily Tasks

- Kiểm tra logs

- Monitor resource usage
- Verify schedulers running
- Check backup status

11.2. Weekly Tasks

- Review error logs
- Clean old temporary files
- Update dependencies
- Performance analysis

11.3. Monthly Tasks

- Security audit
- Database optimization
- Log rotation
- Capacity planning
- Cập nhật Zalo Token: Đăng nhập lại Zalo để lấy token mới (khuyến nghị 1-2 tháng/lần)

12. SCALING

12.1. Horizontal Scaling

- Multiple application instances
- Load balancer
- Database sharding

12.2. Vertical Scaling

- Increase server resources
- Optimize database queries
- Cache frequently accessed data

13. DISASTER RECOVERY

13.1. RTO/RPO (Recovery Time Objective / Recovery Point Objective)

RTO (Recovery Time Objective): 4 giờ

- Thời gian tối đa để khôi phục hệ thống hoạt động bình thường

- Bao gồm: Phân tích lỗi, khôi phục dữ liệu, restart services, kiểm tra chức năng

- Các bước khôi phục:

1. Phân tích nguyên nhân lỗi (30 phút)
2. Khôi phục dữ liệu từ backup (1 giờ)
3. Restart và cấu hình lại services (1 giờ)
4. Kiểm tra và xác nhận hoạt động (1.5 giờ)

RPO (Recovery Point Objective): 1 giờ

- Dữ liệu tối đa có thể mất trong trường hợp sự cố

- Dựa trên tần suất backup và đồng bộ dữ liệu

- Các nguồn dữ liệu:

- Database: Đồng bộ real-time

- Logs: Backup hàng ngày

- Files: Đồng bộ với Google Drive

- CRM data: Đồng bộ 6 giờ/lần

Kế hoạch Disaster Recovery:

1. Immediate Response (0-30 phút): Xác định và phân tích sự cố
2. Recovery Phase (30 phút - 3 giờ): Khôi phục dữ liệu và services
3. Validation Phase (3-4 giờ): Kiểm tra và xác nhận hoạt động
4. Post-Recovery (4+ giờ): Monitoring và tối ưu hóa

13.2. Recovery Steps

1. Assess damage
2. Restore from backup
3. Verify data integrity
4. Restart services
5. Test functionality

14. GOOGLE APPS SCRIPTS

14.1. Tổng quan AppScripts

Hệ thống sử dụng Google Apps Scripts để tự động đồng bộ dữ liệu từ Google Sheets với hệ thống Ralli Chatbot. Các script này chạy trên Google Cloud và gọi API của hệ thống để cập nhật dữ liệu.

14.2. Script đồng bộ dữ liệu Zalo

Mục đích: Đồng bộ danh sách liên hệ Zalo và cấu hình lịch gửi báo cáo

Google Sheets: 1o4PAWEOWKlcRfTaPBzORlkFUPsFVScKRIjuej78PH0s

Các function chính:

1. syncClients(): Đồng bộ danh sách khách hàng
 - Gọi API /process với type 'customer'
 - Đồng bộ thông tin liên hệ từ sheet 'Cài đặt'
2. syncSettings(): Đồng bộ cài đặt và danh sách liên hệ
 - Xử lý danh sách liên hệ từ cột F5
 - Gửi dữ liệu đến API /contacts/sync
 - Cập nhật cấu hình scheduler
3. postContactsToApi(): Gửi danh sách liên hệ đến API
 - Format dữ liệu: province, name, phone, user_id
 - Gọi API /contacts/sync
4. postSchedulerConfig(): Cập nhật cấu hình lịch gửi
 - Đọc thời gian từ B4, ngày trong tuần từ C4
 - Cấu hình lặp lại: Không, Hàng ngày, Hàng tuần, Hàng tháng
 - Gọi API /scheduler/config

14.3. Script đồng bộ dữ liệu sản phẩm

Mục đích: Đồng bộ dữ liệu sản phẩm từ Google Sheets

Google Sheets: 1UWH4a7Efw2Jb5KWcC4xdYldRTX2noe_SdSep8FRaGVY

Function chính:

1. syncDatabase(): Đồng bộ toàn bộ dữ liệu sản phẩm

- Gọi API /process với sheet_id

- Đồng bộ tất cả dữ liệu sản phẩm mới

14.4. Script đồng bộ Catalogs

Mục đích: Đồng bộ từ viết tắt nội bộ và từ đặc biệt

Google Sheets: 1SWk7Q2gaM4_wqXhquq77j28waLISGlVhS9tbp23X3ZQ

Các function chính:

1. syncCatalogs(): Đồng bộ từ viết tắt nội bộ

- Gọi API /specials với resource_type 'catalogs'

- Đồng bộ từ sheet 'catalogs'

2. syncSpecialsKeys(): Đồng bộ từ đặc biệt

- Gọi API /specials với resource_type 'specials_keys'

- Đồng bộ từ sheet 'specials'

14.5. Cách sử dụng AppScripts

Thiết lập ban đầu:

1. Mở Google Sheets tương ứng

2. Vào Extensions > Apps Script

3. Tạo script mới và copy code từ các function trên

4. Lưu và chạy function để test

Chạy tự động:

1. Vào Triggers trong Apps Script

2. Tạo trigger mới cho function cần chạy

3. Chọn tần suất: Time-driven (hàng ngày, hàng giờ, etc.)

4. Lưu trigger

Monitor và debug:

1. Vào Executions để xem log chạy

2. Kiểm tra Logs để debug lỗi

3. Sử dụng sendQuote() để hiển thị thông báo trong Sheets

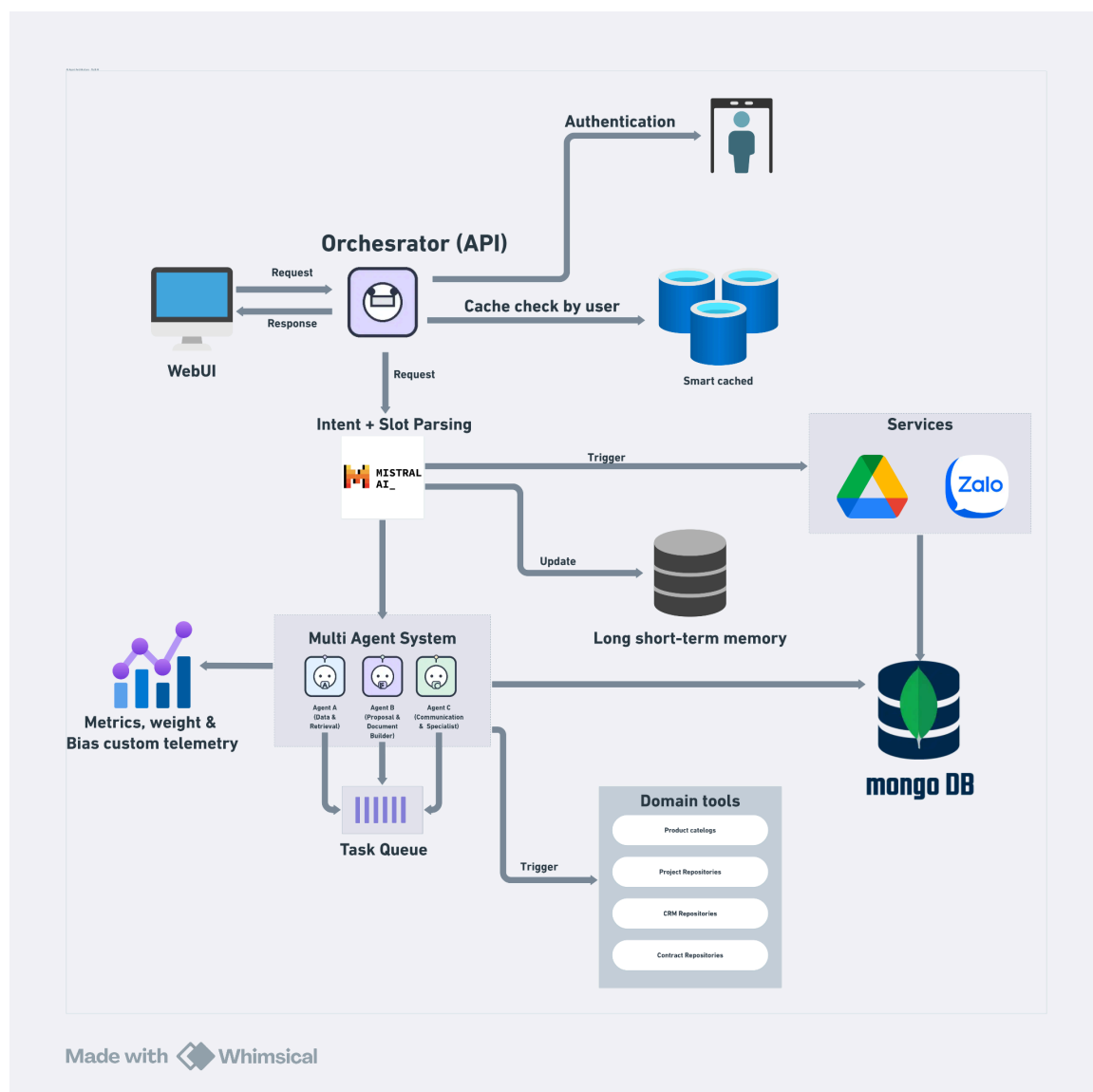
14.6. API Endpoints được sử dụng

- POST /process: Đồng bộ dữ liệu sản phẩm
- POST /contacts/sync: Đồng bộ danh sách liên hệ
- POST /scheduler/config: Cập nhật cấu hình lịch gửi
- POST /specials: Đồng bộ catalogs và specials keys

15. KIẾN TRÚC HỆ THỐNG CHI TIẾT

15.1. Sơ đồ kiến trúc hệ thống

Microservice Architecture:



16. SOURCE CODE & REPOSITORY

16.1. Cấu trúc thư mục

rangdong-chatbot/

```
├── src/           # Backend source code
│   ├── auth/     # Authentication modules
│   ├── core/     # Core functionality
│   └── build_prompts.py # AI prompts
```

```
| | └─ memory_chat.py    # Memory management
| | └─ smart_cache.py    # Caching system
| | └─ logger.py         # Logging system
| └─ schedulers/         # Background tasks
| | └─ data_scheduler.py  # Data synchronization
| | └─ notification_scheduler.py
| | └─ drive_scheduler.py # Google Drive upload
| | └─ crm_scheduler.py   # CRM data sync
| | └─ log_cleanup_scheduler.py
| └─ services/           # Business logic
| └─ repositories/       # Data access layer
| └─ routers/            # API routes
| └─ models/             # Data models
| └─ main.py             # Application entry point
└─ ui/                   # Frontend files
| └─ index.html          # Main UI
| └─ script.js           # JavaScript logic
| └─ styles.css          # Styling
| └─ crm_renders.js       # CRM rendering
| └─ crm_excel.js        # Excel generation
└─ deploy/               # Deployment scripts
| └─ docker-compose.prod.yml # Production compose
| └─ docker-compose.ssl.yml # SSL setup
| └─ start.sh            # Start script
| └─ restart.sh          # Restart script
```

```

|   ├── stop.sh          # Stop script
|   └── watch_log.sh     # Log monitoring
└── processing/          # Data processing
    ├── helpers/         # Utility functions
    ├── logs/            # Log files
    ├── requirements.txt  # Python dependencies
    ├── docker-compose.yml # Development compose
    ├── Dockerfile.backend # Backend container
    ├── Dockerfile.frontend # Frontend container
    ├── nginx.conf        # Nginx configuration
    └── .env              # Environment variables

```

16.2. Naming Convention

Python Files:

- snake_case: data_scheduler.py, memory_chat.py
- Classes: PascalCase: MemoryChatSession, SmartCache
- Functions: snake_case: get_vietnam_time(), build_prompts()
- Variables: snake_case: user_id, message_history

JavaScript Files:

- camelCase: crmRenders.js, databaseManager.js
- Functions: camelCase: renderAsMonth(), getContainer()
- Variables: camelCase: globalData, userInput

Database Collections:

- snake_case: users, projects, crm_data
- Fields: snake_case: user_id, created_at, updated_at

16.3. Hướng dẫn Build & Run

Local Development:

```
bash
```

1. Clone repository

```
git clone <repository-url>
```

```
cd rangdong-chatbot
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Setup environment

```
cp .env.example .env
```

Edit .env with your configuration

4. Start MongoDB

```
mongod --dbpath /path/to/data
```

5. Run application

```
python main.py
```

Staging Environment:

```
bash
```

1. Build Docker images

```
docker-compose build
```

2. Start services

```
docker-compose up -d
```

3. Check logs

`docker-compose logs -f`

Production Environment:

`bash`

1. Deploy with production compose

`docker-compose -f deploy/docker-compose.prod.yml up -d`

2. Setup SSL

`docker-compose -f deploy/docker-compose.ssl.yml up -d`

3. Monitor services

`./deploy/watch_log.sh`

16.4. Coding Guidelines

Python Guidelines:

- Sử dụng type hints: `def get_user(user_id: str) -> User:`
- Docstrings cho functions: `"""Get user by ID."""`
- Error handling với `try/except`
- Logging với `LogManager`
- `Async/await` cho I/O operations

JavaScript Guidelines:

- ES6+ syntax
- JSDoc comments cho functions
- Error handling với `try/catch`
- Consistent indentation (2 spaces)
- Modular code organization

17. API DOCUMENTATION

17.1. Swagger UI

Access: <https://ralliai.rangdong.com.vn:9000/docs>

Available Endpoints:

- Authentication: /auth/login, /auth/me
- User Management: /users/, /users/{id}/role
- Data Processing: /ask, /process, /specials
- Schedulers: /scheduler/health-check, /trigger-*
- Logs: /logs/recent, /logs/cleanup
- System Health: /api/status

17.2. Authentication

JWT Token:

- Algorithm: HS256
- Expiration: 120 minutes (configurable)
- Header: Authorization: Bearer <token>

Example Request:

```
bash
```

```
curl -X GET "https://ralliai.rangdong.com.vn:9000/users/" \  
-H "Authorization: Bearer <your-token>"
```

17.3. Rate Limiting

Current Status: Not implemented

Recommendation: 100 requests/minute per user

Implementation: Can be added with FastAPI rate limiting middleware

17.4. Error Codes

HTTP Status Codes:

- 200: Success

- 400: Bad Request
- 401: Unauthorized
- 403: Forbidden
- 404: Not Found
- 500: Internal Server Error

Custom Error Responses:

json

```
{  
  
  "error": "error_code",  
  
  "message": "Human readable message",  
  
  "details": "Additional information"  
}
```

18. DATABASE DOCUMENTATION

18.1. Mô hình dữ liệu (ERD)

Collections Overview:

users (users)

```
|— _id: ObjectId  
|— username: String (unique)  
|— password_hash: String  
|— role: String (ADMIN/MANAGER/SPECIALIST/MEMBER)  
|— created_at: DateTime  
|— updated_at: DateTime
```

projects (projects)

```
|— _id: ObjectId
```

|— project_id: String (unique)

|— name: String

|— location: String

|— start_date: DateTime

|— end_date: DateTime

|— status: String

|— created_at: DateTime

|— updated_at: DateTime

contacts (contacts)

|— _id: ObjectId

|— province: String

|— name: String

|— phone: String

|— user_id: String

|— created_at: DateTime

|— updated_at: DateTime

crm_data (crm_data)

|— _id: ObjectId

|— month: String

|— year: String

|— metrics: Object

|— created_at: DateTime

|— updated_at: DateTime

18.2. Database Initialization

Setup Script: src/setup_db.py

python

Initialize database

python src/setup_db.py

Create admin user

python src/create_admin.py

MongoDB Commands:

javascript

// Create collections

use rangdong

db.createCollection("users")

db.createCollection("projects")

db.createCollection("contacts")

db.createCollection("crm_data")

db.createCollection("logs")

// Create indexes

db.users.createIndex({"username": 1}, {"unique": true})

db.projects.createIndex({"project_id": 1}, {"unique": true})

db.contacts.createIndex({"user_id": 1})

db.crm_data.createIndex({"month": 1, "year": 1})

18.3. Indexes & Optimization

Critical Indexes:

- users.username (unique)

- projects.project_id (unique)

- contacts.user_id
- crm_data.month_year
- logs.timestamp

Query Optimization:

- Use projection to limit fields
- Implement pagination for large datasets
- Use aggregation pipelines for complex queries
- Monitor slow queries with MongoDB profiler

18.4. Data Retention Policy

Logs: 30 days (configurable)

Projects: Permanent (no automatic deletion)

Users: Permanent (soft delete)

CRM Data: Permanent (historical data)

Contacts: Permanent (business critical)

19. INFRASTRUCTURE / DEVOPS

19.1. Docker Configuration

Backend Dockerfile:

dockerfile

FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "main.py"]

Frontend Dockerfile:

dockerfile

FROM nginx:alpine

COPY ui/ /usr/share/nginx/html/

COPY nginx.conf /etc/nginx/nginx.conf

EXPOSE 80

Docker Compose:

yaml

version: '3.8'

services:

backend:

build: .

ports:

- "9000:8000"

environment:

- MONGODB_URI=mongodb://mongodb:27017

depends_on:

- mongodb

frontend:

build:

context: .

dockerfile: Dockerfile.frontend

ports:

- "9001:80"

mongodb:

image: mongo:4.4

ports:

- "27017:27017"

volumes:

- mongodb_data:/data/db

19.2. CI/CD Pipeline

Current Status: Manual deployment

Recommended: GitHub Actions hoặc GitLab CI

Example GitHub Actions:

yaml

name: Deploy to Production

on:

push:

branches: [main]

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v2

- name: Deploy to server

run: |

docker-compose -f deploy/docker-compose.prod.yml up -d --build

19.3. Backup & Restore Strategy

Database Backup:

bash

Backup MongoDB

```
mongodump --uri="mongodb://admin:password@localhost:27017/rangdong" --out=backup/
```

Restore MongoDB

```
mongorestore --uri="mongodb://admin:password@localhost:27017/rangdong"  
backup/rangdong/
```

File Backup:

- Logs: Daily rotation, 30-day retention
- Configuration: Version controlled in Git
- Uploads: Stored in Google Drive (cloud backup)

Backup Schedule:

- Daily: Database backup
- Weekly: Full system backup
- Monthly: Archive old backups

19.4. Monitoring & Logging

Logging System:

- Framework: Loguru
- Levels: DEBUG, INFO, WARNING, ERROR
- Rotation: Daily files
- Retention: 30 days
- Location: logs/YYYY-MM-DD/

Monitoring Tools:

- System Health: Custom admin panel

- Logs: Real-time via watch_log.sh
- Database: MongoDB monitoring
- External APIs: Health check endpoints

Alerting:

- Email: For critical errors
- Telegram: For system notifications
- Zalo: For business alerts